```
generator = Random.new(1234)
generator.rand          # => 0.1915194503788923

another_generator = Random.new(generator.seed)
another_generator.rand  # => 0.1915194503788923
```

The Random#rand method takes exactly the same arguments as Kernel#rand. You can also get a string of random bytes with the bytes method, which takes one argument: the length of the string.

With the line require random/formatter, you get a number of useful methods mixed in to Random. All of these methods are available as class methods, as in Random.alphanumeric, or as instance methods, as in x.alphanumeric.

- alphanumeric(length = nil)—Returns a randomly generated string with just the characters A-Z, a-z, and 0-9. The argument is the length, and if left off, it defaults to 16.
- base64(length = nil)—Returns a randomly generated base-64 string using the characters in alphanumeric as well as +, /, and =. The length is the length in bytes, not in characters, so the resulting string will be about ⅓ longer than n, since some characters are less than one byte. A version of this method, called urlsafe_base64, uses - and _ instead of + and /. That method also uses = as padding if a second argument is passed with the value true.
- hex(length = nil)—Returns a random hexadecimal string, so the characters are 0-9 and a-f. The resulting string is twice as long as the argument (because the length is in bytes and each character is half a byte), and the default is still 16 (meaning a 32 character string).
- random_bytes(length = nil)—Returns a random binary string. The length is in bytes, and the default is 16.
- uuid—Generates a UUID, with 122 random bits. It corresponds to version 4 of the UUID specification.

One downside of the main Random class is that it is not considered powerful enough for true cryptographic uses. The cyptographically secure randomizers are a little slower, so Ruby provides them in a separate class, called SecureRandom, which you must add to your app with require "securerandom". The only default method of SecureRandom is called bytes, but SecureRandom automatically includes the methods in formatter, so all the above methods are available on a SecureRandom instance. There is not a way to specify the seed of a SecureRandom instance, presumably that would be insecure.

Regular Expressions

Ruby uses the Onigmo regular expression library, which is an extension of the Oniguruma regular expression engine.[2,3] Onigmo offers a number of extensions beyond traditional Unix regular expressions. Most of these extensions are written between the characters (? and). The parentheses that bracket these extensions are groups, but they do not necessarily generate backreferences—meaning that they do not necessarily set the values of \1, $1, and so on.

The sequence, (?#COMMENT) inserts a comment into the pattern. The content is ignored during pattern matching. Commenting complex regular expressions can be as helpful as commenting other complex code.

2. https://github.com/k-takata/Onigmo
3. https://github.com/kkos/oniguruma